Columbia University

APMAE4302

Methods in computational science

Homework 4

Author:
Matías I. Inostroza
(mii2106)

May, 2026

1 Section 1

The original equations are:

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T \tag{1}$$
$$-\nabla^2 \mathbf{v} + \nabla p = T\hat{\mathbf{j}} \tag{2}$$
$$\nabla \cdot \mathbf{v} = 0 \tag{3}$$

Let's define:

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}) \tag{4}$$
$$\omega \hat{\mathbf{k}} = \nabla \times \mathbf{v} \tag{5}$$

First, the velocity is defined in terms of ψ by writing the components explicitly:

$$\mathbf{v} = \nabla \times \psi \hat{\mathbf{k}} = \nabla \times (0, 0, \psi) = (\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x}, 0) \tag{6}$$
$$\rightarrow \nabla \cdot \mathbf{v} = \frac{\partial^2 \psi}{\partial x \partial y} - \frac{\partial^2 \psi}{\partial y \partial x} = 0 \tag{7}$$

Thus, the incompressibility condition is automatically satisfied. Then, substituting (4) in (5)

$$\omega \hat{\mathbf{k}} = \nabla \times (\nabla \times (\psi \hat{\mathbf{k}})) = \nabla(\nabla \cdot (\psi \hat{\mathbf{k}})) - \nabla^2 (\psi \hat{\mathbf{k}}) \tag{8}$$

Since the problem is two dimensional, that means $\psi(x, y)$ then $\nabla \cdot (\psi \hat{\mathbf{k}}) = \frac{\partial \psi}{\partial z} = \mathbf{0}$, and considering that $\hat{\mathbf{k}}$ is constant

$$\omega \hat{\mathbf{k}} = (-\nabla^2 \psi) \hat{\mathbf{k}} \tag{9}$$

And comparing the $\hat{\mathbf{k}}$ component, the third equation is obtained:

$$\omega = -\nabla^2 \psi \ \textbf{Answer.} \tag{10}$$

From equation (2), applying $\nabla \times ()$ on both sides, and considering the identity $\nabla \times (\nabla a) = 0$, being a a scalar field

$$-\nabla \times (\nabla^2 \mathbf{v}) + \nabla \times (\nabla p) = \nabla \times T\hat{\mathbf{j}} \tag{11}$$
$$-\nabla \times (\nabla^2 \mathbf{v}) = \nabla \times T\hat{\mathbf{j}} \tag{12}$$

The curl and Laplacian con commute, then, considering equation (5)

$$-\nabla^2(\nabla \times \mathbf{v}) = \nabla \times T\hat{\mathbf{j}} \tag{13}$$
$$-\nabla^2(\omega\hat{\mathbf{k}}) = \nabla \times T\hat{\mathbf{j}} \tag{14}$$

The right hand side $\nabla \times T\hat{\mathbf{j}} = (\frac{\partial T}{\partial z}, 0, \frac{\partial T}{\partial x})$, but since the problem is two dimensional, then the variation of T with respect to z is zero, or $\frac{\partial T}{\partial z} = 0$. Also, $\hat{\mathbf{k}}$ is considered to be constant, then:

$$-\nabla^2(\nabla \times \mathbf{v}) = \nabla \times T\hat{\mathbf{j}} \tag{15}$$
$$-\nabla^2(\omega)\hat{\mathbf{k}} = \frac{\partial T}{\partial x}\hat{\mathbf{k}} \tag{16}$$

Comparing the $\hat{\mathbf{k}}$ component of the equation, second equation is obtained:

$$-\nabla^2\omega = \frac{\partial T}{\partial x} \textbf{ Answer.} \tag{17}$$

2 Section 2

a) After running the three PETSc option files, Table 1 summarizes the results:

Options file	**SNES its**	**KSP its**	**SNES time (s)**	**FLOPs**	**Rate (Mflop/s)**
`direct`	1	1	4.0272	2.54×10^8	63
`split_direct`	1	1	5.2119	1.44×10^8	28
`split_mg`	1	7	3.8605	8.26×10^8	214

Table 1: Convergence and runtime comparison of PETSc solver configurations for the biharmonic problem.

All three PETSc configurations converged successfully. The direct and split-direct configurations required only one Krylov iteration because the linear systems were solved using direct LU factorization. The multigrid fieldsplit configuration required more Krylov iterations, but achieved the shortest solve time due to the efficiency of the multigrid preconditioner.

The `split_mg` configuration also achieved the highest floating-point rate, indicating more efficient hardware utilization during the solve phase.

b) After running the three option files with Firedrake, table 2 shows the results:

Solver option	**SNES its**	**KSP its**	**SNES time (s)**	**FLOPs**	**Rate (Mflop/s)**
`monolithic`	1	1	4.3293	2.11×10^9	488
`fieldsplit_direct`	1	1	3.2560	1.68×10^9	516
`fieldsplit_mg`	1	7	3.9000	2.58×10^9	661

Table 2: Convergence and runtime information for the Firedrake biharmonic solver configurations.

All three Firedrake solver configurations converged successfully and produced comparable numerical solutions. The monolithic and `fieldsplit_direct` configurations required only one Krylov iteration due to the use of direct LU factorization, while the multigrid configuration required more iterations but achieved competitive runtimes.

Among the tested configurations, `fieldsplit_direct` achieved the lowest `SNESSolve` time, whereas `fieldsplit_mg` achieved the highest floating-point rate at the expense of a larger number of FLOPs. Overall, the `fieldsplit_direct` configuration provided the best balance between robustness and computational efficiency.

c)

Quantity	**PETSc C**	**Firedrake**
Mesh size	513×513	512×512
Solver configuration	`split_direct`	`fieldsplit_direct`
SNES iterations	1	1
KSP iterations	1	1
Total runtime (s)	6.09	15.65
SNESSolve time (s)	5.21	3.26
FLOPs	1.45×10^8	1.68×10^9
Rate (Mflop/s)	28	516
Relative error (vorticity)	5.20×10^{-5}	5.72×10^{-5}
Relative error (streamfunction)	1.17×10^{-4}	1.46×10^{-4}

Table 3: Comparison between the PETSc C and Firedrake implementations using the direct fieldsplit solver configuration.

Both implementations converged successfully and produced comparable numerical errors. The PETSc C implementation achieved a significantly lower runtime, while Firedrake required substantially more floating-point operations.

The main difference arises from the discretization strategy and software framework used by each implementation. The PETSc C code uses a structured finite-difference discretization with low level assembly, whereas Firedrake employs finite elements,

mixed function spaces, and automatically generated variational kernels, leading to larger algebraic and assembly costs.

Although Firedrake achieved a higher floating point rate, the larger computational workload dominated the runtime.

Overall, both implementations produced similar numerical accuracy, but the PETSc C implementation was computationally more efficient for this benchmark problem.

3 Section 3

For this part, the following modifications were introduced to the original `biharm.py` code, including the definition of the temperature field ass:

$$T(x, y) = (1 - y) + A\cos(\pi x) \tag{18}$$

And a RHS function as:

$$f(x) = \frac{\partial T}{\partial x} \tag{19}$$

```
A = 0.1
T = Function(V,name="Temperature")
T.interpolate((1.0-z)+A*cos(pi*x))
f.interpolate(T.dx(0))
```

No error analysis was performed for this part, since the problem is not posed as a manufactured solution test. Instead, the objective is to use the prescribed temperature field to compute the forcing term $f = \partial T/\partial x$ and solve the corresponding biharmonic streamfunction vorticity system. Therefore, the result is assessed qualitatively through the Paraview visualization of the temperature field, streamfunction contours, and velocity field.

Figure 1 shows the temperature field with the stream function as contour plot (most likely circle shaped), and the velocity vectors in grey color.

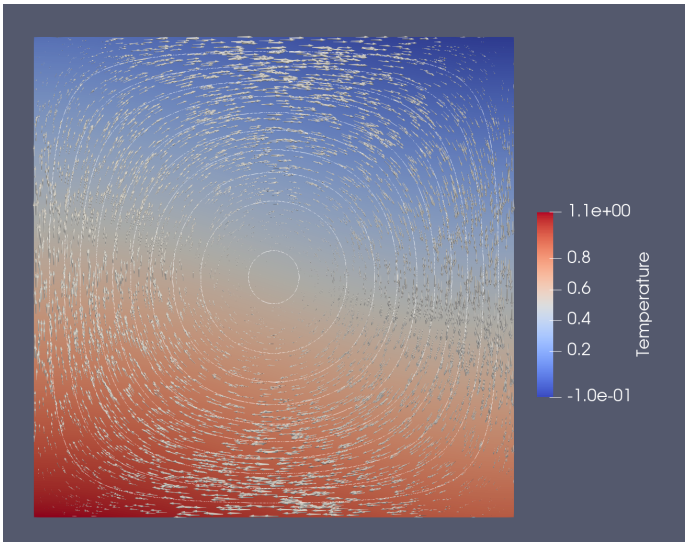

Figure 1: Solution from Firedrake

4 Section 4

Starting from the temperature equation

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T \tag{20}$$

the equation is multiplied by a test function q and integrated over the domain Ω:

$$\int_\Omega q \frac{\partial T}{\partial t} \, d\Omega + \int_\Omega q \, \mathbf{v} \cdot \nabla T \, d\Omega = \frac{1}{Ra} \int_\Omega q \, \nabla^2 T \, d\Omega \tag{21}$$

Integrating the diffusion term by parts gives

$$\int_\Omega q \frac{\partial T}{\partial t} \, d\Omega + \int_\Omega q \, \mathbf{v} \cdot \nabla T \, d\Omega + \frac{1}{Ra} \int_\Omega \nabla q \cdot \nabla T \, d\Omega = 0 \tag{22}$$

where the boundary integral vanishes due to the prescribed boundary conditions. Therefore, the weak form used in the Firedrake implementation is

$$F_T(T; q) = \int_\Omega q \, T_t \, d\Omega + \int_\Omega q \, \mathbf{v} \cdot \nabla T \, d\Omega + \frac{1}{Ra} \int_\Omega \nabla q \cdot \nabla T \, d\Omega \tag{23}$$

This is the line code implemented in firedrake

```
FT = (q*T_dot*dx+ q*dot(vel, grad(T))*dx + (1.0/Ra)*inner(
    grad(T), grad(q))*dx)
```

a) The boundary conditions were implemented as:

```
bcs = [DirichletBC(ME.sub(0), 1.0, 3),    # T(x,0,t)=1
       DirichletBC(ME.sub(0), 0.0, 4),    # T(x,1,t)=0
       DirichletBC(ME.sub(1), 0.0, "on_boundary"),   # omega=0
       DirichletBC(ME.sub(2), 0.0, "on_boundary"),   # psi=0]
```

And for computation of the Nusselt number, the following equation was implemented:

$$Nu = -h\frac{\int_0^l \partial_z T(x, z = h)dx}{\int_0^l T(x, z = 0)dx} \tag{24}$$

with $h = 1$. This definition was taken from Lecture 20. Unfortunately, the expression provided in the homework statement did not produce reasonable values of the Nusselt number greater than 1. Using the homework definition, the computed value of Nu always converged to approximately 1, and I could not figure out the error.

Figure 2 shows the Nusselt number for $Ra = 10^2$.

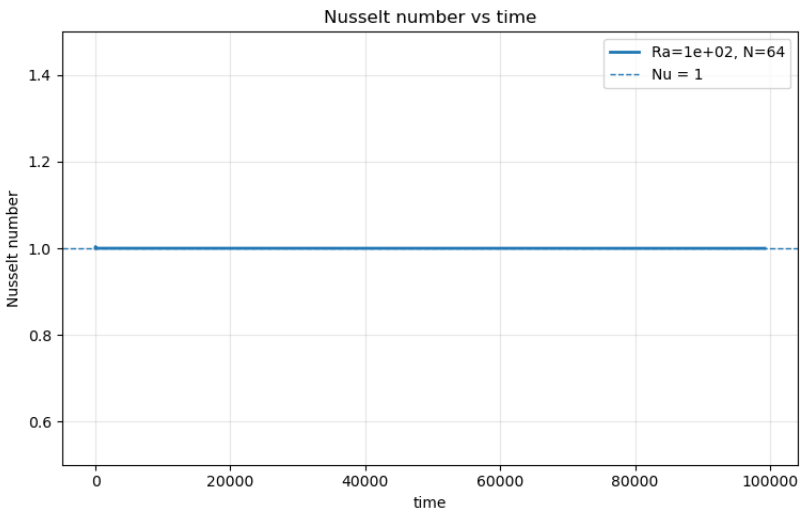

Figure 2: Nusselt number for $Ra = 10^2$

For $Ra = 10^2$ on a 64×64 Q1 mesh, the computed Nusselt number converged to $Nu \approx 1$

Since the Nusselt number measures the total heat flux relative to the purely conductive heat flux, $Nu = 1$ indicates that heat transport is dominated by conduction and that there is no significant convective enhancement. Therefore, the steady state for $Ra = 10^2$ is non convective.

b) For larger Rayleigh numbers, the evolution of Nu is shown in Figure 3. The final values of Nu at $t = 100000$ s are reported in Table 4.

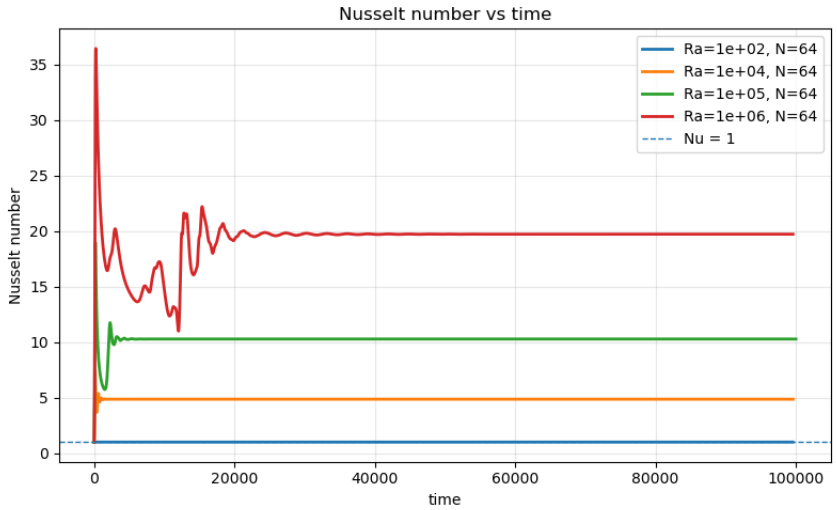

Figure 3: Ra analysis

Table 4: Final Nusselt number for different Rayleigh numbers using a 64×64 mesh.

Rayleigh Number (Ra)	Final Nusselt Number (Nu)
10^2	0.999
10^4	4.859
10^5	10.274
10^6	19.711

c) Figure 4 shows the mesh convergence of the final Nusselt number for $Ra = 10^4$. As the mesh is refined from 16×16 to 128×128, the computed value of Nu increases and approaches the Blankenbach benchmark value $Nu = 4.884$. The coarse 16×16 mesh underestimates the heat flux, while the 64×64 and 128×128 meshes give values very close to the benchmark. This indicates that the numerical solution is converging with mesh refinement.

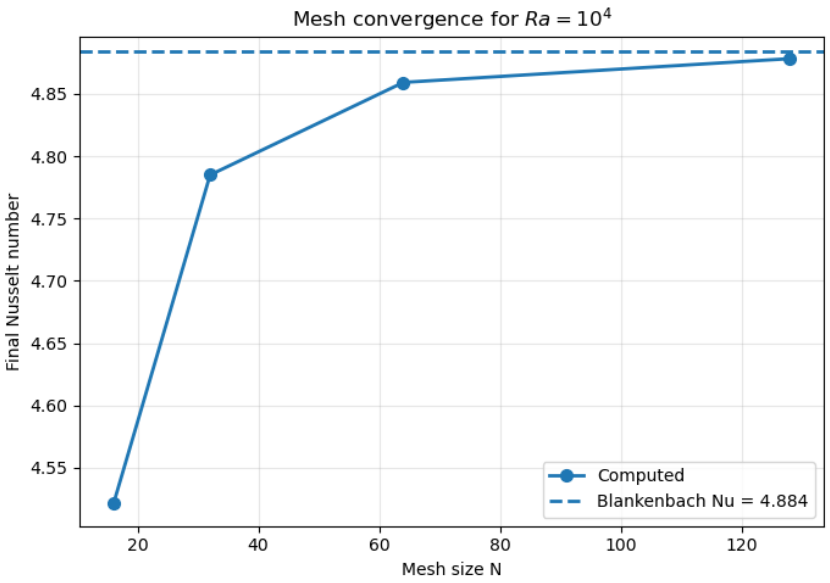

Figure 4: Convergence analysis